MINI PROJECT REPORT

On

Smart Traffic Routing and Telemetry System

By

ANAND MORE (BECOC34)

Under the guidance of

(Prof. Nitanjali Mane)

Department of Computer Engineering

Pimpri Chinchwad College of Engineering and Research, RavetPlot No: B, Sector No. 110, gate no 1, Laxminagar, Ravet, Pune,Pin.: 412101

Savitribai Phule Pune University [2025-2026]

Department of Computer Engineering

Pimpri Chinchwad College of Engineering and Research,

Ravet

CERTIFICATE

This is certify that **Anand Ankush More** from Fourth Year Computer Engineering has successfully completed her mini project work titled "**Smart Traffic Routing and Telemetry System**" at Pimpri Chinchwad College of Engineering and Research, Ravet in the partial fulfillment of the Bachelor's Degree in Engineering.

Mrs. Nitanjali Mane	Dr. Vijay A. Kotkar	Prof. Dr. Harish Tiwari
Guide	**Head of Department**	**Principal**

ABSTRACT

The rapid expansion of complex networking infrastructures necessitates the deployment of highly scalable and agile network architectures. This report presents the design, development, and implementation of a mock Software-Defined Networking (SDN) ecosystem utilizing the Nexus Controller and Mininet network emulation framework. The core deliverable is the "Nexus NOC (Network Operations Center) Dashboard," a full-stack telemetric interface providing network administrators with real-time visualization of OpenFlow parameters. The system aggregates high-frequency load-balancing metrics—such as switch latency, dynamic bandwidth fluctuations, and packet routing table entries—and visualizes active topological failover sequences. By decoupling the control plane from the data plane, this project effectively demonstrates how modern enterprise networks can proactively mitigate congestion and dynamically enforce shortest-path algorithmic routing without administrative intervention.

Keywords: *Software-Defined Networking (SDN), OpenFlow Protocol, Nexus Controller, Mininet, Network Telemetry, Load Balancing, React.js*

Contents

Chapter 1

Introduction

Traditional network topologies are heavily reliant on highly coupled, proprietary hardware where the control and data planes reside within the same physical device. This paradigm inherently limits scalability and agile network management. Software-Defined Networking (SDN) revolutionizes this by abstracting the routing intelligence into a centralized, programmable controller.

This project explores the practical application of SDN principles by engineering an automated OpenFlow-based network. The primary objective is twofold: to simulate dynamic traffic congestion and intelligent rerouting within a virtualized network environment, and to build an enterprise-grade Network Operations Center (NOC) dashboard capable of visualizing live telemetry data to facilitate administrative network diagnostics.

Chapter 2

System Architecture and Methodology

The architectural design of the proposed system leverages a multi-tier framework, seamlessly integrating network emulation with modern web technologies.

2.1 The Data Plane: Mininet Emulation

The foundational physical layout is instantiated via the Mininet framework (`topo.py`). The topology consists of multi-layered networking devices mapping eight OpenFlow-enabled switches (e.g., $S_1, S_2, ...S_8$) interconnected via 14 discrete active links, extending to 16 virtual hosts. This environment executes localized load generation sequences, establishing baseline TCP/UDP traffic and artificial link saturation to induce latency peaks.

2.2 The Control Plane: Nexus SDN Controller

At the apex of the hierarchy operates the Nexus SDN controller (`nexus_controller.py`). Operating purely via the OpenFlow v1.3 protocol, the controller calculates optimal routing paths across the directed acyclic graph representing the Mininet topology. Upon detecting link thresholds exceeding maximum bandwidth capacities across the primary vector (Path A: $S_1 \rightarrow S_5 \rightarrow S_8$), the controller autonomously calculates and deploys updated OpenFlow instruction sets to redirect traffic through a failover vector (Path B: S_4).

2.3 Application Layer: NOC Telemetry Dashboard

The application plane consists of a full-stack telemetry visualization engine:

- **Backend Simulator Server:** A robust Node.js and Express RESTful API operating natively on Port 5001. A specialized `SDNTrafficSimulator` class abstracts chronological network matrices, injecting realistic statistical variances mimicking controller polling structures.

- **Frontend User Interface:** A high-performance React application incorporating Framer Motion for kinetic feedback. It translates localized JSON telemetry into responsive AreaCharts and complex SVG-driven topographical mapping arrays.

Chapter 3

Results and Deliverables

The implemented system successfully achieves all primary operational requirements:

3.1 Dynamic Topology Visualization

The web interface natively translates textual topology structures into interconnected graphical nodes. Upon synthetic link congestion, the controller's failover action is visually expressed; primary links are greyed out, and alternate pathways immediately glow to indicate active packet traversal, giving administrators unambiguous operational context.

3.2 Flow Table Analytics

OpenFlow rules installed on arbitrary switches are securely polled and displayed dynamically. The dashboard presents critical L2/L3 matching criteria (e.g., `in_port=1, eth_type=0x0800`), priority integers, designated packet output ports (`output:3`), and total packet execution payloads, enabling granular network diagnostics and verification of controller actions.

3.3 Real-Time Metrics Monitoring

Key Performance Indicators (KPIs) such as global bandwidth utilization (in Mbps) and localized switch latency (in ms) are polled and streamed into historical coordinate planes. The interface immediately reflects severe latency perturbations and standardizes the visualization across customizable epoch windows.

Chapter 4

Future Enhancements

While currently operational as an end-to-end simulated engine, several enterprise-level advancements are under consideration:

1. **Machine Learning (ML) Integration:** Integration of predictive analytic algorithms (e.g., Random Forest or LSTM models) deployed on the backend to predict congestion events vectorially prior to their initiation.
2. **WebSocket Architecture:** Refactoring the HTTP polling cycle into persistent, bi-directional WebSocket connections to facilitate zero-latency real-time controller updates directly to the graphical interface.
3. **Hardware Deployment:** Transposing the Nexus controller logic from the simulated Mininet environment to physical bare-metal OpenFlow-enabled Cisco or Juniper switches to validate physical throughput parameters.

Chapter 5

Conclusion

The design and successful implementation of the Nexus NOC Center underscore the vast potential of Software-Defined Networking. By decoupling network intelligence into a centralized pythonic instance and building a professional, decoupled visualization array, this project proves that complex routing protocols can be demystified and effectively managed by modern administrative interfaces. The resulting application acts as a foundational framework for scalable, agile, and robust network infrastructures.